

# MANUAL TRD

## Operación, Inventario de Plantillas y Reconocimiento de Problemas

Esta guía acompaña a *plantilla.pdf*. Los algoritmos aparecen en el **mismo orden** que en la plantilla; el marcador **(0)/(1)** indica la base de indexación, igual que en los comentarios *Uso[0]/Uso[1]* de cada header. Cuando no aplica, no se marca.

## 0. Operación en Competencia

### 0.1 Protocolo de la Máquina Única

Hay 1 computadora para 3 personas; el recurso escaso es el tiempo de máquina.

- **Papel primero:** nadie programa sin tener escritas las transiciones del DP, la condición de parada o la red de flujo con sus capacidades.
- **Regla de los 15 minutos:** si llevas 15 min atascado en un bug, cede la máquina y depura en papel mientras otro avanza.
- **Implementación por pares:** uno escribe y otro revisa el código en paralelo. Reduce errores de índices, tipos y olvidos de casos borde.
- **Antes de cada submit:** corre el checklist de la sección 0.4 y casos de prueba extra (no solo los de la muestra).

### 0.2 Lectura y Gestión de Tiempo

- **Leer TODOS los problemas** en la primera pasada, antes de escribir código.
- No priorizar por “los más resueltos”: el número de submits no mide dificultad. Un problema ignorado puede ser el más fácil.
- Anota por problema: idea, técnica, complejidad estimada y prioridad. Empieza por los fáciles confirmados.
- **Pausas cortas** (2–3 min) al estancarte para refrescar la mente; rota de problema y de máquina.

### 0.3 Roles

- **JuanJoe-18:** Sorting, Greedy, Geometría, Grafos, Range Queries y Strings. Identificación de estructuras y reducciones.
- **Botinix:** implementador principal; velocidad en templates y resolución de bugs difíciles.
- **JuanchoKing:** Chess, Matemáticas, Teoría de Números y Combinatoria. Reducciones abstractas.

### 0.4 Checklist Pre-Submit

- **Overflow:** si sumas o multiplicaciones pueden superar  $2 \times 10^9$ , usa *ll* y escribe *1LL \* a \* b*.
- **INF:** para caminos mínimos usa *LINF* (1e18); *INF* (1e9) solo si los pesos lo permiten.
- **Múltiples testcases:** limpia estructuras globales solo hasta *N*; no reasignes con *memset* de tamaños gigantes.
- **Indexación:** verifica la base (0)/(1) de cada estructura que uses (marcada en el inventario).
- **Casos borde:**  $N = 1, 2$ ; grafo desconexo (¿imprime  $-1$ ?); árbol en línea recta; cadena vacía o con un solo carácter; polígono degenerado; valores extremos (0, negativos).
- Imprime con *'\n'*, nunca *endl*.

### 0.5 Stress Testing

Cuando hay WA y un brute force confiable, verifica contra casos aleatorios pequeños. Bloque base:

```
import random, subprocess

def gen():
    n = random.randint(1, 8)          # tamanos pequenos (brute rapido)
    a = [random.randint(1, 20) for _ in range(n)]
    return f"{n}\n{' '.join(map(str, a))}\n"

for it in range(2000):
    data = gen()
    sol = subprocess.check_output("./sol", input=data, text=True).strip()
    brute = subprocess.check_output("./brute", input=data, text=True).strip()
    if sol != brute:
        print("FALLO en:" + data)
        print("sol:", sol, "\nbrute:", brute)
        break
    else:
        print("OK: no se encontraron diferencias")
```

Uso:

- Compila tu solución y un brute force (misma lectura de entrada, misma salida) como *sol* y *brute* (en Windows, con *sol.exe* y *brute.exe*).
- **Qué variar en *gen()*** según el tipo de problema:
  - *Arrays:* valores negativos y positivos, duplicados, extremos, tamaños 1 y 2.
  - *Grafos/árboles:* árboles aleatorios (nodo *i* conectado a un padre  $< i$ ), grafos con pesos, desconexos, con ciclos.
  - *Strings:* un solo carácter repetido, patrones que se solapan, cadenas cortas.
  - *Grillas:* *R*, *C* pequeños, con obstáculos, bordes de tamaño 1.
  - *Flujo:* capacidades 0, cuellos de botella, demandas mínimas.
- Si falla, reduce el caso a uno mínimo y depura con él.
- Para salidas de precisión (double), compara con tolerancia, no con igualdad.
- Si no aparece fallo en 2000 iteraciones, sube las iteraciones o cambia la semilla.

## 1. Main y Configuración (*src/main.cpp*)

- *fastio:* *sync\_with\_stdio* apagado.
- *all*, *rall*, *pb*.
- typedefs *ll*, *ld*, *pii*, *vi*, *vll*.
- constantes *INF*, *LINF*, *MOD*, *EPS*.
- *rand\_val(a, b)*: entero aleatorio.
- *pbd*s (*ordered\_set*) incluido; *solve()* con bucle de *T* testcases.

Tips y errores:

- Usa `rand_val` para bases aleatorias de hash y test aleatorio; el `rng` local no colisiona con el de `rq.hpp`.
- No uses `endl` (flushea). Usa `'\n'`.
- Los pragmas de optimización están comentados; actívalos solo si el juez lo permite y el problema lo exige.

## 2. Grafos (*graph.hpp*)

### Inventario (orden de la plantilla):

#### Grillas

- `dx4, dy4, dx8, dy8` (1).
- `isValid` (1).

#### Internas

- `Edge<T>`.

#### Conjuntos

- `DSU` (0): `find`, `unite`, `component_size`, `members`, `comps`, `get_components`.

#### Estructura

- `Graph<T>` (1): `add_directed_edge`, `add_undirected_edge`.

#### os mínimos

- `Dijkstra<T>` (1): `restore_path`, `invalid_edges`.
- `MultiStateDijkstra<T>` (1): multi-estado.
- `ZeroOneBfs<T>` (1): pesos  $\{0, 1\}$ .
- `FloydWarshall<T>` (1).
- `BellmanFord<T>` (1): ciclo negativo.
- `Bipartite<T>` (1).

#### Orden

- `TopoSort<T>` (1): vacío si hay ciclo.
- `DirectedCycle<T>` (1).
- `UndirectedCycle<T>` (1).

#### Componentes

- `TarjanSCC<T>` (1).
- `BridgeTree` (1): puentes, articulación, árbol de componentes 2-arista-conexas.

#### MST

- `Kruskal<T>` (1): devuelve índices de aristas.

#### s especiales

- `EulerianCircuit<T>` (1).
- `Hamiltonian` (1,  $N \leq 15$ ).
- `FunctionalGraph` (1): k-ésimo sucesor, ciclos.
- `DominatorTree` (1).

#### LCA

- `LCA<T>` (1): binary lifting.

#### Booleano

- `TwoSat` (1): cláusulas, implicación, xor, equivalencia.

#### arejamiento

- `BipartiteMatcher` (1): Kuhn.
- `HopcroftKarp` (1).

#### Clique

- `MaxClique` (0,  $N \leq 60$ ).

#### Flujo

- `Dinic` (0): `restore_path`.
- `DinicLB` (1): cotas inferiores.

- `MCMF` (0): potenciales.

### Reconocimiento (patrón → algoritmo):

- “Menor costo de  $A$  a  $B$ ” → *Dijkstra* (pesos  $\geq 0$ ) o *ZeroOneBfs* (solo  $\{0, 1\}$ ).
- “Con hasta  $K$  descuentos/boletos/combustible” → *MultiStateDijkstra*: estado (*nodo*, *recurso*), respuesta =  $\min_{rem} dist[t][rem]$ .
- “Costos negativos o detectar ciclo negativo” → *BellmanFord*.
- “Todos los pares,  $N \leq 400$ ” → *FloydWarshall*.
- “Conectar todo al mínimo costo” → *Kruskal*.
- “Orden de tareas con dependencias” → *TopoSort* (si hay ciclo devuelve vacío).
- “Recorrer cada arista una vez” → Euleriano; “cada nodo una vez” → *Hamiltonian* (solo  $N \leq 15$ ).
- “Cada nodo apunta a un único nodo” (funcional) → *FunctionalGraph*: k-ésimo sucesor, ciclos, *dist\_to\_cycle*.
- “Asignar cada  $X$  a un  $Y$  sin repetir” → *BipartiteMatcher/HopcroftKarp*. König: **min vertex cover** = **max matching**; **max independent set** =  $N - \text{matching}$ . Dilworth: **máxima anticadena** = **min path cover** (con matching duplicando vértices).
- “Proyectos con ganancia + costos + dependencias obligatorias” (Project Selection) → min-cut:  $S \rightarrow$  proyecto (ganancia), herramienta  $\rightarrow T$  (costo), proyecto  $\rightarrow$  herramienta ( $\infty$ ); respuesta =  $\Sigma \text{ganancias} - \text{mincut}$ .
- “Variables binarias con restricciones por pares” → *TwoSat*. “No ambas activas” =  $(A \Rightarrow \neg B) \wedge (B \Rightarrow \neg A)$ ; “al menos una” =  $(\neg A \Rightarrow B) \wedge (\neg B \Rightarrow A)$ ; “iguales” = ambas implicaciones.
- “Grafo implícito” (estados como nodos, transiciones como aristas) → BFS / 0-1 BFS / *Dijkstra*.
- “Minimizar el máximo / maximizar el mínimo / menor tiempo tal que...” → **búsqueda binaria sobre la respuesta** con *check* monótono (a menudo *Dijkstra* o *greedy*).
- **Grillas**: aplanar con  $id = r * C + c$ ; grilla 4-conexa es **bipartita** por  $(r+c) \bmod 2$  (dominós  $\rightarrow$  matching; König para bloquear celdas); capacidad por celda  $\rightarrow$  dividir  $u$  en  $u_{in} \rightarrow u_{out}$  con capacidad  $K$  y  $u_{out} \rightarrow v_{in}$  con  $\infty$ .

### Errores y tips:

- *Graph*, *Dijkstra*, *LCA*, *TwoSat*, *BridgeTree*, *DinicLB*, *DominatorTree* son **1-based**; *DSU*, *Dinic*, *MCMF*, *MaxClique* son **0-based**.
- *DSU* es 0-based: si el problema da nodos  $1..n$ , usa *DSU*  $dsu(n+1)$  (el nodo 0 queda aislado).
- *MultiStateDijkstra*: *rem* es el recurso **restante**; usar  $K$  descuentos termina en  $rem=0$ .
- *invalid\_edges* requiere *e.id* != -1; asigna ids al crear las aristas.
- *Kruskal* devuelve índices de aristas, no de nodos.
- *Dijkstra* con pesos negativos falla; usa *BellmanFord*.
- *MaxClique* es 0-based y exponencial: solo  $N \leq 60$  con bitsets.
- *EulerianCircuit*: usa *e.id* para no repetir aristas; en no dirigido el circuito tiene  $E/2 + 1$  vértices.

### 3. Árboles (*tree.hpp*)

#### Inventario (orden de la plantilla):

##### Internas

- *TreeEdge*<*T*>.

##### Árbol base

- *Tree*<*T*> (1): raíz, euler *tin/tout*, *sz*, *depth*, *get\_diameter* con camino, *get\_centers*.
- *all\_farthest\_distances*: distancia máxima por nodo.
- *BinaryLifting*<*T*> (1): *lca*, *kth\_ancestor*, *path\_sum*, *path\_max*, *path\_min*, *dist\_weighted*, *kth\_node\_on\_path*, *is\_ancestor*.
- *EulerTourTree*<*T*> (1): mapea subárbol a [*tin*, *tout*].
- *HLD* (1): *values\_on\_edges* nativo, *process\_path*, *process\_subtree*.

##### Centroid

- *CentroidPathProcessing* (1): Query-then-Add.
- *CentroidTree*<*T*> (1): distancia al nodo activo más cercano.
- *CentroidBinarySearch* (1): nodo oculto interactivo.

##### s y caminos

- *DSUOnTree* (1): small-to-large.
- *VirtualTree* (1): con lambdas de LCA.
- *TreeHashing* (1): isomorfismo.
- *RerootingDP* (1).
- *FastLCA* (1): Euler+RMQ  $O(1)$ .
- *MoOnTrees* (1).

#### Reconocimiento:

- “Ancestro común / distancia en árbol” → *BinaryLifting* ( $O(\log N)$ ) o *FastLCA* ( $O(1)$  si hay muchas consultas).
- “Consultas sobre caminos (suma/mín/máx/asignación)” → *HLD* + *LazySegTree*; con pesos en aristas usa *values\_on\_edges = true*.
- “Consultas sobre subárboles” → *EulerTourTree* + *Fenwick/SegTree*.
- “Para cada nodo como raíz, calcula  $X$ ” → *RerootingDP* (merge + extend + finalize).
- “Contar/optimizar caminos por el árbol (ej. exactamente  $K$  aristas)” → *CentroidPathProcessing*.
- “Actualizar y consultar el nodo activo más cercano / suma de distancias” → *CentroidTree* (inyecta la función de distancia).
- “Encontrar un nodo oculto con consultas interactivas” → *CentroidBinarySearch*.
- “Estadísticas de colores por subárbol” → *DSUOnTree*.
- “DP sobre un subconjunto de nodos (con LCA de por medio)” → *VirtualTree*.
- “Frecuencias de colores/valores en caminos  $u-v$ ” → *MoOnTrees*.
- “¿Dos árboles son isomorfos?” → *TreeHashing*.
- Diámetro y centros → *Tree.get\_diameter/get\_centers*.

#### Errores y tips:

- Todo en *tree.hpp* es **1-based**.
- *Tree* requiere *init(root)* antes de *tin* y *tout*; *all\_farthest\_distances* usa el diámetro y asume el árbol inicializado.

- *HLD* con *values\_on\_edges*: el LCA se excluye automáticamente; no consultes el nodo raíz.
- *RerootingDP*: *merge* debe ser asociativo/conmutativo para el paso top-down (usa prefijos/sufijos si no es invertible).
- *CentroidPathProcessing*: modifica *PathState*, *GlobalState*, *extend*, *query\_and\_merge*, *add\_to\_global*, *clear\_global*.
- *CentroidTree*: modifica *State*, *NEUTRAL*, *merge*, *combine*.
- *MoOnTrees*: implementa *toggle(u)* antes de *solve*.
- *VirtualTree*: recibe *get\_lca*, *is\_anc*, *get\_dist*, y orden por *tin* como lambdas.

### 4. Range Queries (*rq.hpp*)

#### Inventario (orden de la plantilla):

- *SparseTable* (0): consultas estáticas idempotentes (min/max/gcd).
- *FenwickTree* (1): sumas + update puntual.
- *IterativeSegTree* (0): suma/min/max + update puntual.
- *RecursiveSegTree* (0): *Node*+merge, *find\_first*.
- *MaxSubarraySegTree* (0): máxima suma de subsegmento contiguo.
- *LazySegTree* (0): *add\_range*, *set\_range*, *get*.
- *DynamicSegTree* (1..max): sparse sin compresión.
- *PersistentSegTree* (0): versiones históricas.
- *FenwickTree2D* (1): submatrices + update puntual.
- *MergeSortTree* (0): conteo  $\leq k$  en rango + update.
- *SqrtDecomposition* (0).
- *Treap* (0): orden estadístico.
- *ImplicitTreap* (0): secuencia con reverse.
- *Mo* (0): consultas offline.
- *MoWithUpdates* (0): Mo 3D.
- *RollbackMo* (0): solo add.

#### Reconocimiento:

- RMQ estático → *SparseTable*.
- Suma con updates puntuales → *Fenwick* (más rápido y simple que el segment tree).
- Operación no idempotente o con updates → segment tree; si la operación es compleja, redefine *Node*+merge en *RecursiveSegTree*.
- “Máxima suma de subsegmento en un rango” → *MaxSubarraySegTree* (nodo con *sum*, *pref*, *suff*, *ans*).
- “Suma/asignación sobre un rango + consulta” → *LazySegTree*.
- “Índices enormes sin comprimir” → *DynamicSegTree*.
- “Consultas sobre versiones anteriores” (ej.  $k$ -ésimo menor en rangos) → *PersistentSegTree*.
- “Contar elementos  $\leq k$  en un rango (con updates)” → *MergeSortTree*.
- “Consultas offline de rangos” → *Mo*; con updates → *MoWithUpdates*; si solo se agrega (sin remover fácil) → *RollbackMo*.
- “Secuencia dinámica con insert/borrar/revertir” → *ImplicitTreap*.
- “ $k$ -ésimo menor de un multiset dinámico” → *Treap*.

#### Errores y tips:

- **Fenwick:**  $idx = 0$  entra en **bucle infinito** ( $0 \ \&\ -0 = 0$ ). Si tus datos son 0-based, haz  $ft.add(x+1, v)$  y  $ft.query(l+1, r+1)$ .
- 0-based: *SparseTable*, *IterativeSegTree*, *RecursiveSegTree*, *LazySegTree*, *MaxSubarraySegTree*, *PersistentSegTree*.
- 0-based: *MergeSortTree*, *SqrtDecomposition*, *Mo*, *MoWithUpdates*, *RollbackMo*, *ImplicitTreap*. 1-based: *FenwickTree*, *FenwickTree2D*. *DynamicSegTree* usa  $1 \dots \max$ .
- *LazySegTree*:  $update(type\ 1=add, 2=set)$ ; tras un  $set\_range$ , un  $add\_range$  posterior suma sobre el valor seteado.
- *PersistentSegTree*:  $update$  devuelve la nueva raíz; guarda cada versión.
- *Mo*: las queries se ordenan internamente;  $add/remove$  deben ser  $O(1)$ .
- *Treap.kth\_element* es 0-based y devuelve *LINF* fuera de rango.

#### Variaciones:

- Desacoplar *Node+merge* (max subarray ya en plantilla): nunca toques los índices del árbol.
- Lazy para otra operación: edita *merge* y cómo *apply\_add/apply\_set* combinan el valor del nodo.
- Segment tree + Euler (subárboles) o + *HLD* (camino): los rangos que produce el árbol alimentan la estructura 1D.

## 5. Strings (*string.hpp*)

#### Inventario (orden de la plantilla):

##### Utilidades

- *to\_lower*, *to\_upper*, *trim*, *replace\_all*, *split*, *join*.

##### s y hashing

- *prefix\_function* (KMP).
- *z\_function*.
- *remove\_all\_occurrences* (KMP con pila).
- *rabin\_karp* (0): hash simple.
- *manacher*: radios de palíndromos.
- *StringHash* (0): doble módulo, bases configurables o aleatorias.

##### Estructuras

- *Trie*.
- *AhoCorasick* (0): multi-patrón.
- *SuffixAutomaton* (0).
- *SuffixArray* (0): con LCP RMQ.
- *PalindromicTree* (0).

#### Reconocimiento (cuál usar):

- *KMP* (*pi*): periodo de una cadena, prefijo que es sufijo, autó-mata de un patrón.
- *Z*: para cada sufijo, LCP con el inicio de la cadena; más intuitivo que KMP para modificar.
- *remove\_all\_occurrences*: eliminar un patrón incluyendo ocurrencias encadenadas (ej. “aaa” con “aa”).
- *StringHash*: comparar substrings en  $O(1)$ ; con *true* usa bases aleatorias (inmune a colisiones forzadas en jueces adversariales).
- *rabin\_karp*: posiciones de un patrón; base fija  $\Rightarrow$  preferir *StringHash* si hay riesgo de colisiones.

- *AhoCorasick*: **múltiples patrones** en un texto (conteo y primera aparición).
- *SuffixAutomaton*: substrings distintos, ocurrencias de un patrón, primera aparición.
- *SuffixArray*: sufijos ordenados + LCP entre cualquier par en  $O(1)$  (repeticiones, substrings comunes, conteos con lcp).
- *Manacher*: radio del palíndromo más largo en cada centro.
- *PalindromicTree* (eertree): todos los palíndromos distintos, sus ocurrencias y primera posición.
- *Trie*: autómatas de prefijos, inserciones/búsquedas.

#### Errores y tips:

- *StringHash* y *rabin\_karp* asumen minúsculas ( $s[i]-'a'+1$ ); adapta la constante si hay otros caracteres.
- *manacher* devuelve radios; el máximo radio es la longitud del palíndromo más largo.
- *SuffixArray* agrega internamente un '\$' a la cadena;  $p[0]$  es el sufijo “\$”.
- *PalindromicTree*: *count\_palindromes* son los distintos; llama *compute\_occurrences()* antes de leer *occ*.
- *StringHash*(*s*, *true*) genera bases distintas por proceso: no compares hashes de instancias diferentes.

## 6. Programación Dinámica (*dp.hpp*)

#### Inventario (orden de la plantilla):

- *LiChaoTree*: mínimo de  $m \cdot x + c$  ( $x$  arbitraria).
- *dnc\_dp*: divide & conquer para  $dp[i][j] = \min_{k < j} (dp[i-1][k] + cost(k+1, j))$  con decisión monótona.
- *knuth\_dp*: optimización de Knuth para DP de intervalos.
- *DigitDP*.
- *sos\_dp*: suma sobre submáscaras.
- *lis*: con reconstrucción.
- *bitset\_knapsack*.
- *WQS*: Alien’s trick.
- *CHTDeque*: pendientes monótonas.

#### Marco teórico (DP = caminos sobre un DAG):

- Estado = nodo; transición = arista; DP = caminos sobre el DAG inducido. Mínimo costo = camino más corto; máximo beneficio = camino más largo; conteo = número de caminos.
- Ante un DP “duro”, responde en papel: (1) **eje de avance**: ¿qué variable avanza monótono (índice, suma, tiempo, bits)?; (2) **memoria mínima**: ¿qué información del pasado condiciona el futuro?; (3) **decisiones legales**: ¿qué transiciones e invariantes hay?
- **Permutaciones vs combinaciones**: contar composiciones que suman  $S$  con 1D ( $dp[s] = \sum_c dp[s-c]$ , distingue orden) vs 2D ( $dp[i][s]$  con índice de moneda, orden fijo, colapsa combinaciones).

#### Reconocimiento (qué optimización):

- “ $dp[i] = \min_j (m_j \cdot x_i + b_j)$ ”  $\rightarrow$  *LiChaoTree* ( $x$  arbitraria) o *CHTDeque* (pendientes y  $x$  monótonos).
- “Particionar en  $K$  grupos /  $dp[i][j] = \min_k (dp[i-1][k] + C(k, j))$  con *opt* monótono”  $\rightarrow$  *dnc\_dp*.
- “Intervalos:  $dp[i][j] = \min_k (dp[i][k] + dp[k][j]) + cost$ ”  $\rightarrow$  *knuth\_dp*.
- “Contar números que cumplen condiciones de dígitos”  $\rightarrow$  *DigitDP* (recuerda *tight* y *lz*).

- “Suma sobre subconjuntos de máscaras”  $\rightarrow$  *sos\_dp* ( $O(N2^N)$ ).
- “Subsecuencia creciente más larga (con reconstrucción)”  $\rightarrow$  *lis*.
- “¿Subconjunto suma exactamente  $S$ ?”  $\rightarrow$  *bitset\_knapsack* (rápido,  $S \leq 10^5$  fijo).
- “Exactamente  $K$  elementos/particiones con función convexa”  $\rightarrow$  *WQS* (Alien’s trick).
- “Para cada nodo como raíz”  $\rightarrow$  *RerootingDP* (tree.hpp).
- **Greedy**: si no hay subestructura que exija DP (actividades, intervalos, monedas canónicas), ordena y elige localmente; verifica con un contraejemplo antes de descartar.

#### Errores y tips:

- *dnc\_dp*: *cost* debe evaluarse en  $O(1)$  (usa prefijos); para particionar en  $K$  grupos, *dp\_prev*[0] = 0; 0-based.
- *knuth\_dp*: requiere quadrangle inequality en *cost*.
- *DigitDP*: *memo* se resetea en cada *solve*; *lz* evita contar ceros a la izquierda.
- *WQS*: *check(mid).se*  $\geq K$  decide la búsqueda.
- *CHTDeque.query(x)* exige  $x$  no decreciente; usa *query\_bs* si  $x$  es arbitraria.
- *bitset\_knapsack*: *MAX\_SUM* está fijo en  $10^5$ ; si el objetivo es mayor, redimensiona la constante.

## 7. Matemáticas (*math.hpp*)

#### Inventario (orden de la plantilla):

- *modpow*.
- *ext\_gcd*.
- *modinv*.
- *crt*: dos congruencias.
- *solve\_crt\_array*.
- *Sieve*: primos, *spf*, *mu*, *phi*, *factorize*.
- *Combinatorics*: *nCr*, *catalan*.
- *gauss*: sistemas lineales (double).
- *multiply* (FFT): polinomios enteros.
- *multiply\_mod* (NTT).
- *nim\_game*.
- *sum\_n*, *sum\_squares*, *sum\_cubes*, *geom\_sum*.
- *Matrix*: multiplicación y potencia modular.
- *berlekamp\_massey*.
- *PollardRho*: *is\_prime* (Miller-Rabin), *factorize* hasta  $2^{64}$ .
- *Lucas*.
- *xor\_convolution*, *or\_convolution*, *and\_convolution* (FWHT).
- *floor\_sum*.

#### Reconocimiento:

- “Recurrencia lineal grande / número de caminos en  $k$  pasos”  $\rightarrow$  *Matrix.power*.
- “Conjeturar la recurrencia de una secuencia”  $\rightarrow$  *berlekamp\_massey*.
- “ $nCr$  con  $n$  enorme y módulo primo pequeño”  $\rightarrow$  *Lucas*.
- “Multiplicar polinomios”  $\rightarrow$  *multiply* (FFT, enteros) o *multiply\_mod* (NTT). “Convoluciones XOR/OR/AND”  $\rightarrow$  FWHT.

- “Sistema de congruencias”  $\rightarrow$  *crt/solve\_crt\_array*.
- “Factorizar enteros de hasta  $2^{64}$  / ¿es primo?”  $\rightarrow$  *PollardRho*.
- “Möbius / totient / factorizar muchos valores”  $\rightarrow$  *Sieve*.
- “Juego de Nim”  $\rightarrow$  xor de pilas (*nim\_game*).
- “Sumatorias cerradas”  $\rightarrow$  *sum\_n* etc.; “ $\sum [(ai + b)/m]$ ”  $\rightarrow$  *floor\_sum*.
- “Sistema de ecuaciones lineales”  $\rightarrow$  *gauss*.

#### Errores y tips:

- *modinv* exige  $\gcd(a, m) = 1$  (*assert*).
- *crt* con congruencias inconsistentes devuelve  $\{-1, -1\}$ ; verificalo.
- NTT requiere tamaño potencia de 2 y que el módulo tenga la raíz ( $1e9+7$  usa raíz primitiva 3).
- FFT usa *double*: para coeficientes modulares muy grandes prefiere *multiply\_mod*.
- *Matrix* multiplica en  $O(r \cdot c^2)$ : cuidado con matrices grandes.
- *Lucas* preprocesa factoriales hasta  $p - 1$  mód  $p$ .

## 8. Geometría (*geometry.hpp*)

#### Inventario (orden de la plantilla):

- *cmp*: comparación con EPS.
- *Point<T>*: operadores + orden lexicográfico.
- *dot*, *cross*, *cross(p,a,b)*, *norm2*, *norm*.
- *sgn*, *orient*.
- *on\_segment*.
- *segment\_intersect*.
- *polygon\_area\_2*: doble del área.
- *boundary\_points*: Pick.
- *point\_in\_polygon*.
- *convex\_hull*: preserva colineales.
- *closest\_pair*.
- *Event1D*, *Event2D*: sweep line.
- *polar\_sort*.
- *dist\_to\_line*.
- *dist\_to\_segment*.
- *line\_intersect*.
- *rotating\_calipers*: diámetro.
- *Circle*.
- *circle\_line\_intersect*.
- *circle\_circle\_intersect*.
- *Halfplane*.
- *halfplane\_intersection*.
- *Point3D*.
- *dist\_point\_plane*, *dist\_point\_line*.
- *line\_plane\_intersect*.

#### Reconocimiento:

- “Orientación / colinealidad”  $\rightarrow$  *cross*, *orient* (enteros).
- “¿Punto dentro de un polígono? / área / puntos del retículo”  $\rightarrow$  *point\_in\_polygon*, *polygon\_area\_2*, *boundary\_points* (Pick:  $A = I + B/2 - 1$ ).
- “Cáscara convexa / envolvente”  $\rightarrow$  *convex\_hull*; “diámetro de un convexo”  $\rightarrow$  *rotating\_calipers*.

- “Par de puntos más cercano” → *closest\_pair*.
- “Intersección de segmentos/rectas/círculos” → *segment\_intersect*, *line\_intersect*, *circle\_\**.
- “Región factible de restricciones lineales (semiplanos)” → *halfplane\_intersection*.
- “Intervalos que se solapan / área de unión de rectángulos” → sweep line con *Event1D/Event2D* + segment tree.
- “Ordenar puntos por ángulo” → *polar\_sort*.

#### Errores y tips:

- Usa enteros (*ll*) mientras puedas; con doubles usa *cmp* y *EPS*.
- *convex\_hull* preserva colineales en el borde (usa *cross < 0*); para cáscara estricta cambia a *cross <= 0*.
- *polygon\_area\_2* devuelve el doble del área (divide entre 2).
- *line\_intersect* no es válida si las rectas son paralelas.
- *halfplane\_intersection* devuelve vacío si la región es ilimitada o no existe; el lado válido es la izquierda de la recta orientada.
- *point\_in\_polygon* devuelve 0 (fuera), 1 (borde), 2 (dentro).

## 9. Bits (*bits.hpp*)

#### Inventario (orden de la plantilla):

- *check\_bit*, *set\_bit*, *clear\_bit*, *toggle\_bit*, *lsb* (bit *i* 0-based).
- *count\_bits*, *count\_bits\_ll*, *highest\_bit*, *ctz*.
- *iterate\_submasks*: itera submáscaras (sin la vacía).
- *gospers\_hack*: máscaras con *k* bits.
- *XorBasis*: base lineal xor.
- *BitTrie*: máximo xor con borrado.

#### Reconocimiento:

- “Máximo xor de un subconjunto / ¿se puede formar *k*?” → *XorBasis*.
- “Máximo xor de un valor contra un conjunto (dinámico)” → *BitTrie*.
- “DP sobre submáscaras” → *iterate\_submasks* (total  $O(3^N)$ ) o *sos\_dp* ( $O(N2^N)$ ).
- “Subconjuntos de tamaño exacto *k*” → *gospers\_hack*.
- “Comparar multiconjuntos de rangos” → *ZobristHash* (extra).

#### Errores y tips:

- Escribe *1LL* « *i* (no *1* « *i*) con  $i \geq 31$ .
- *BitTrie* usa 30 bits por defecto (hasta  $\approx 10^9$ ); sube a 60 para valores hasta  $10^{18}$ .
- *iterate\_submasks* no incluye la submáscara vacía (procesa *sub = 0* aparte si la necesitas).
- *highest\_bit* devuelve -1 para 0; *ctz* devuelve 64 para 0.

## 10. Extra (*extra.hpp*)

#### Inventario (orden de la plantilla):

- *ordered\_set* (0): pbds, k-ésimo menor, *order\_of\_key*.
- *ordered\_multiset* (0): pbds, permite duplicados.
- *custom\_hash*: splitmix64 anti-colisión.
- *MeetInTheMiddle*.
- *ZobristHash* (0).
- *BitsetReachability* (1).
- *compress\_coords* (0).

#### Reconocimiento:

- “k-ésimo menor dinámico / posición de un elemento” → *ordered\_set*.
- “*unordered\_map* con claves numéricas (hackeadas)” → *custom\_hash*.
- “ $N \approx 36 \dots 42$ : demasiado para  $2^N$ , sin subestructura DP” → *MeetInTheMiddle* (dos mitades de  $2^{N/2}$ , buscar con *lower\_bound*).
- “Comparar multiconjuntos de rangos en  $O(1)$ ” → *ZobristHash* (prefijos XOR con hashes aleatorios por valor).
- “Alcanzabilidad todos-pares en grafos densos ( $N^3/64$ )” → *BitsetReachability*.
- “Comprimir coordenadas para segment tree/BIT/HLD” → *compress\_coords*.

#### Errores y tips:

- *ordered\_multiset*: inserta *{valor, id}* para permitir duplicados; *find\_by\_order* devuelve *{valor, id}*.
- *BitsetReachability* es 1-based.
- *compress\_coords* devuelve ids 0-based en orden creciente de valor.
- *MeetInTheMiddle.subset\_sum* asume valores no negativos para el objetivo exacto (verifica el problema).

## 11. Recomendaciones

- Tras cada competencia, actualiza la guía y la plantilla: si un problema requirió modificar un algoritmo, documenta la variación.
- Practica a conciencia los algoritmos que más se modifican en vivo: *Dijkstra* (multi-estado), *HLD*, segment trees (Node/merge y lazy), DP (identificación de recurrencia) y flujo (reducciones a min-cut).
- El inventario refleja el orden de *plantilla.pdf*; si cambias los headers, recompila la plantilla y revisa que la guía siga alineada.
- Usa la guía como apoyo teórico y de navegación; el código vive en la plantilla.